

Git

Nitharshini V S

This paper presents Git as a distributed version control system and demonstrates its main operations through a practical repository exercise. The report covers repositories, staging, commits, branches, merging, merge conflicts, rebasing, remotes, push and pull, relog, and stashing. A Driverless Taskphase repository was used to demonstrate these operations in a real local-to-remote workflow. The practical sequence included creating feature branches, performing a fast-forward merge, deliberately producing and resolving a merge conflict, using stash, pushing to GitHub, fetching a remote update, and pulling the update into the local branch. The existing Task 1 solutions were also verified through Git history and status commands.

Git, version control, commit, branch, merge, rebase, remote repository, relog, stash

I. I. WHAT IS GIT?

Git is a popular version control system. It was created by Linus Torvalds in 2005, and has been maintained by Junio Hamano since then. It is used for: Tracking code changes, tracking who made changes, Coding collaboration. Key git concepts are-

Repository: A folder where Git tracks your project and its history.

Clone: Make a copy of a remote repository on your computer.

Stage: Tell Git which changes you want to save next.
Commit: Save a snapshot of your staged changes.

Branch: Work on different versions or features at the same time

Merge: Combine changes from different branches.

Pull: Get the latest changes from a remote repository.

Push: Send your changes to a remote repository.

While setting git up, there are different stages- mainly three stages. First, the working directory which is completely local where all the work done. Second is the staging area in which we tell git what are the files which are ready to be committed. Last is when we commit the changes in which git takes the snapshots of the changes made. Git allows you to see the full history of every commit. You can revert back to any previous commit.

II. II. GIT CONCEPTS AND COMMANDS

A. A. Repository, Staging, and Commits

A repository is the project directory tracked by Git. When a file is changed, Git detects the modification. A commit records a snapshot of the staged state and is identified by a hash. The commit message describes the purpose of the change. In the practical repository, the initial history contained commits such as “initial commit” and later program updates. The command `git log --oneline` was used to inspect this history. All the commits remain, they don’t get rewritten. A commit is like a save point in your project.

It records a snapshot of your files at a certain time, with a message describing what changed. You can always go back to a previous commit if you need to. Here are some key commands for commits:

`git commit -m "message"` - Commit staged changes with a message

`git commit -a -m "message"` - Commit all tracked changes (skip staging)

`git log` - See commit history



B. B. Branching and Merging

A branch is a separate line of development. It allows a feature to be developed without immediately changing the main branch. This helps in the simultaneous development of different features in the same project by branching out of the main branch and developing and in the end merging them together.

Merging is the process where two branches can be combined back once the work on both are done. Sometimes it so happens that the changes made, that are merged actually oppose each other. So this creates something called merging conflict and has to be resolved.



In the demonstration, a feature branch was created and was checked out. The `hello.txt` file was edited in the branch to “hello from the feature”. Then the changes were staged and committed. Then the master branch was checked out and the same was done where the file was edited to be “hello from the master”. This instance shows an example of a conflict where the same file has been edited in conflicting ways and git now wouldn’t know which version to keep. Hence when we merge them, git shows a conflict and it must be resolved first. For that, it’s essential for the user to resolve the conflict by choosing either one or something which combines both. Once the conflict is resolved, merging becomes possible.

C. C. Rebase

Rebase moves or reapplies a sequence of commits onto a new base. It can produce a more linear history by avoiding an unnecessary merge commit. However, because rebasing rewrites commit history, it should be used carefully, especially for commits already shared with others. Rebase basically

- [1] W3Schools, “Introduction to Git,” W3Schools Git Tutorial.